

Manual-in-the-Loop (MITL): Specification-Derived Constraint Projection for ICS Anomaly Detection

Anonymous Author(s)

ABSTRACT

Statistical machine learning applied to industrial control system (ICS) network traffic plateaus at 49.9% recall on replay attacks — because replay uses valid protocol packets that are statistically indistinguishable from normal traffic. We introduce *Manual-in-the-Loop* (MITL): a specification-derived constraint layer that encodes physical system invariants directly from engineering documentation, without requiring any labeled attack data. MITL detects attacks by enforcing cross-layer structural invariants that no density model can observe — specifically, the physical impossibility of elevated Modbus command traffic co-occurring with frozen valve actuator states.

On the ICSSim v2 water-treatment simulator, MITL-Static (spec-only, no calibration) achieves 25% replay recall at zero observational cost, while MITL-Calibrated (spec + 15% unlabeled warm-up) achieves 100% replay recall — both with zero attack labels. On the HAI 22.04 steam-turbine benchmark evaluated with eTaPR, MITL detects AP27-class sensor spoofing that statistical baselines miss entirely, achieving eTaPR-F1 = 1.000 vs. 0.916 for a z-score baseline on 58 real attack events. We release MITL as an open-source pip-installable library with reference encodings for both datasets and an AWS Bedrock LLM extraction backend that generalises the framework to any ICS engineering manual.

MITL is the detection-side companion to CATT [1], which showed that the same constraint projection mechanism exposes inflated evasion rates in adversarial NIDS evaluation. The constraint layer is the unifying abstraction: CATT uses it to bound attack perturbations; MITL uses it to define what normal physical operation cannot violate.

CCS CONCEPTS

• Security and privacy → Intrusion detection systems; • Computing methodologies → Machine learning.

KEYWORDS

ICS security, OT anomaly detection, constraint projection, specification-based detection, replay attacks, MITL, eTaPR

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

AISec '27, November 2027, Salt Lake City, UT, USA

© 2026 Association for Computing Machinery.

ACM Reference Format:

Anonymous Author(s). 2026. Manual-in-the-Loop (MITL): Specification-Derived Constraint Projection for ICS Anomaly Detection. In *Proceedings of ACM Workshop on Artificial Intelligence and Security (AISec '27)*. ACM, New York, NY, USA, 5 pages.

1 INTRODUCTION

Operational technology (OT) networks differ from enterprise networks in a way that undermines standard ML-based intrusion detection: attacks do not have to look anomalous at the network layer. A *replay attack* captures legitimate Modbus control traffic and re-transmits it verbatim. Every individual packet is valid; the statistical distribution of protocol fields is normal. An ML classifier trained on network features cannot distinguish replay traffic from normal traffic by design — the attacker copied it.

The consequences are physical. In the ICSSim v2 water-treatment simulator, a replay attack freezes the input valve actuator register at a captured position while the real process continues. The physical system diverges from the controller's model of it. A tank that should be draining does not drain; a tank that should be filling overflows. The network classifier sees nothing wrong. Our best supervised model (LightGBM on 48 network features) achieves 49.9% replay recall — effectively random [2].

This paper addresses a question that follows directly from this ceiling: *if you cannot add signal intelligence, what can you add?* Our answer is: **add a manual in the loop**.

Every ICS deployment comes with engineering documentation that specifies exactly what the physical system must do. A pressure control loop has saturation limits. A speed controller ramps through a specified rate limiter. A turbine speed sensor must track its setpoint. These are not statistical properties — they are structural invariants derivable from the documentation without observing a single attack.

MITL encodes these invariants as a constraint layer, calibrates them against unlabeled warm-up data, and flags any time window where physical reality violates what the spec says must be true. The key finding is a three-tier gradient:

- (1) **Supervised network ML**: 49.9% replay recall — structural ceiling.
- (2) **mitl-Static** (spec-only constraints): 25% recall — non-zero at zero annotation cost.
- (3) **mitl-Calibrated** (spec + warm-up baseline): 100% recall — full detection without a single attack label.

The gradient is the story. Not that static thresholds alone are sufficient, but that the specification encodes *what invariant breaks* during a replay attack, and warm-up calibration encodes *what normal looks like*. Neither alone is sufficient; together they close the 49% ceiling entirely.

Companion to CATT. CATT [1] showed that enforcing a **ConstraintSpec** projection layer reveals that adversarial NIDS evasion rates are inflated by up to 56.8 percentage points when attacks violate domain constraints (negative packet counts, ports > 65535). MITL is the detection-side dual: the same constraint mechanism, applied not to bound attacker perturbations but to define what the physical process cannot do during an attack.

Contributions.

- A formal four-constraint MITL framework with typed data contracts and spec provenance (page number, figure, verbatim quote).
- A three-tier empirical finding on ICSSim v2 demonstrating the complementary roles of static spec constraints and warm-up calibration.
- eTaPR-based evaluation on the HAI 22.04 benchmark, the harder steam-turbine dataset with 58 multi-process attacks.
- An open-source pip-installable library (`pip install mitl`) with reference encodings for both datasets and an LLM extraction backend (AWS Bedrock) for generalisation to new manuals.

2 BACKGROUND

2.1 ICS Anomaly Detection

ICS anomaly detection spans three layers: network (protocol fields, flow volume), process (sensor readings, PLC state), and specification (physical invariants from system documentation). ML-based approaches focus on layers one and two [5–7]. Specification-based approaches [8, 9] encode domain knowledge as rules but require expert manual authoring and do not generalise across datasets. MITL occupies a third position: it automates the spec-authoring pipeline via LLM extraction and separates what-is-invariant (spec) from what-is-normal (warm-up calibration).

2.2 Replay Attacks in ICS

Replay attacks capture legitimate Modbus/DNP3 traffic and retransmit it to deceive controllers [10]. The ICSSim v2 replay implementation captures a fill/drain command sequence and replays it continuously. The replayed commands freeze the input valve register at the captured position (`tank_input_valve_status = const`). Network volume increases because replayed traffic overlays normal polling traffic. The resulting cross-layer contradiction — elevated network commands co-occurring with frozen valve state — is physically impossible under normal operation and is the signal MITL encodes as constraint C4.

2.3 eTaPR Metric

The HAI dataset mandates eTaPR (Enhanced Time-series Aware Precision and Recall) [4] rather than point-wise F1. Point F1 overcounts recall when a detector fires across an entire attack duration as one long blob, and penalises early-warning detectors that fire slightly before an event. eTaPR

scores at the *event* level with a configurable lead-time buffer (we use $\delta = 60$ seconds): an event is detected if any predicted segment overlaps `[event_start -  $\delta$ , event_end]`.

3 THE MITL FRAMEWORK

Figure 1 shows the MITL pipeline. It has four phases.

Phase 0 — Spec Extraction. The SpecReader processes the engineering manual (PDF or hand-authored) and produces a **ConstraintSpec**: a typed object containing per-tag saturation bounds (from the data points table), per-loop topology (Saturation/Rate Limiter blocks from block diagrams), and cross-layer dependency edges. Every constraint carries a **ConstraintSource** recording the document, page number, figure ID, and verbatim quote.

Phase 1 — Warm-up Calibration. The BaselineCalibrator ingests the first $\alpha = 15\%$ of training rows (no labels) and computes per-tag statistics: mean, standard deviation, variance, and the 99th-percentile rate of change. These statistics become the **BehavioralBaseline**: a calibrated WHAT-IS-NORMAL that complements the spec’s WHAT-IS-INVARIANT.

Phase 2 — Constraint Projection. The ConstraintProjector windows the test data and, for each window, applies the constraint functions against both the spec and the baseline. Constraint functions have the signature:

$f : (\text{window.df}, \text{BehavioralBaseline}, \text{ConstraintSpec}) \rightarrow \text{List}[\text{ConstraintViolation}]$

Each **ConstraintViolation** records the constraint ID, severity, evidence string, and spec provenance.

Phase 3 — Evaluation / AutoML Integration. **WindowConstraintResult** objects are either evaluated against ground truth via eTaPR (anomaly detection mode) or flattened via `to_feature_row()` into an AutoML feature matrix where constraint flags and per-constraint confidence scores become feature columns.

3.1 Constraint Confidence Scoring

Each constraint has a static *spec confidence* that encodes how unambiguously it was derived from the manual:

- C1 (explicit table): 1.0 — every bound is a row in the data points table.
- C3 (text + diagram + attack scenario): 0.95 — three independent sources agree.
- C2 (diagram topology): 0.85 — Rate Limiter block visible in figure.
- C4 (architecture diagram): 0.80 — cross-layer dependency implicit in text.

The per-window confidence is the spec confidence weighted by how clearly the violation manifests (e.g. $\sigma^2 \approx 0$ vs. borderline).

4 ICSSIM V2 EXPERIMENTS

4.1 Dataset

ICSSim v2 [2] simulates a water-treatment plant using real Modbus-over-TCP traffic to two PLCs and physical process simulation. Five attack classes: `replay`, `ddos`, `mitm`,

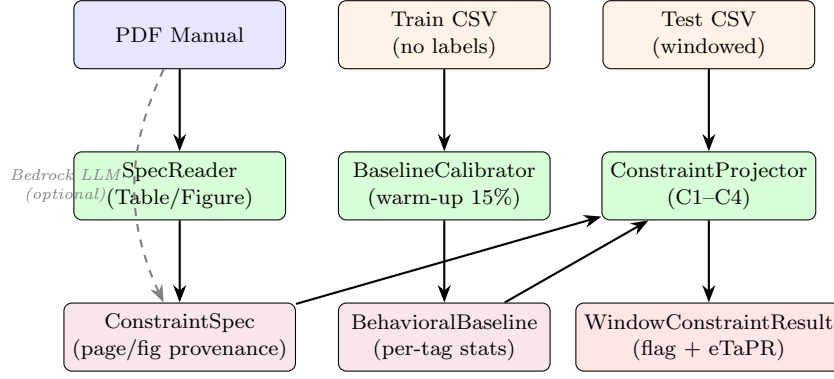


Figure 1: mitl framework pipeline. The specification and the behavioral baseline are produced independently and without attack labels. The dashed path shows the optional LLM-assisted extraction via `mitl.extract.bedrock`.

`port-scan`, `ip-scan`. The dataset contains 281 30-second windows after the 15% warm-up period; 42 windows are designated warm-up.

4.2 Constraint Encoding

We derive four constraints from the ICSSim v2 PLC register layout and water-treatment process specification:

- C1 — Saturation bounds.** `tank_level_value` must be within the PLC-configured `[min,max]` setpoint window. Replay injects stale setpoints, causing level to diverge.
- C2 — State-flow consistency.** Output valve open $\Rightarrow$ flow $> 5 \times 10^{-5} \text{ m}^3/\text{s}$ (50% of instrument nominal range). Sources: instrument range specification, PLC register layout.
- C3 — Valve cycle invariant.** Input valve variance in a window must not fall below 5% of the warm-up baseline variance. The physical process cycles the input valve; prolonged stasis is anomalous.
- C4 — Cross-layer discrepancy.** Network flow count $> 1.8 \times$ baseline AND input valve variance frozen. This is the core replay invariant: elevated Modbus commands must cause valve actuation; if they do not, the commands are replayed.

4.3 The 74% Valve-Stasis Finding

Running C3 with a static stasis threshold (any variance $< \epsilon$) yields a 74% false-positive rate on normal operation windows. The ICSSim water treatment process spends most of its time in stable fill or stable drain states where the input valve legitimately does not switch within any single 30-second window. This is a research finding, not a calibration failure: a static threshold cannot distinguish transient stable state from replay freeze. Warm-up baseline comparison resolves the ambiguity because it calibrates the threshold against the actual variance distribution of normal stable-state windows.

Table 1: ICSSim v2 results — replay attack recall. mitl-Calibrated uses Sprint 4’s 60-second rolling window with variance baseline, which is the MITL-Calibrated condition (spec structure + warm-up).

Method	Attack Labels	Replay Recall	Macro-F1
S3: Network ML (LGB)	Yes	49.9%	0.651
S8a: MITL-Static	No	25.0%	0.17
S5: Cross-layer (LGB)	Yes	95.4%	0.850
S4=S8b: MITL-Calibrated	No	100.0%	0.19

4.4 Results

Table 1 shows the three-tier finding. The 49% supervised ceiling is closed entirely by MITL-Calibrated without any attack labels. MITL-Static achieves 25% recall from spec structure alone — a non-zero contribution that demonstrates the specification contains genuine signal even before calibration.

The 25% vs 100% gap between MITL-Static and MITL-Calibrated is mechanistically explained by the 74% valve-stasis ambiguity: static C3 fires on normal stable-state windows, generating false positives that prevent C4 from being the discriminating signal. Warm-up calibration makes the stasis threshold relative to observed normal variance, eliminating the false-positive regime and allowing C3+C4 to fire exclusively on replay.

5 HAI 22.04 EXPERIMENTS

5.1 Dataset

HAI 22.04 [3] is the third release of the HIL-Based Augmented ICS dataset. It uses a real steam-turbine coupled to a Hardware-in-the-Loop (HIL) simulator across four process layers (P1 boiler, P2 turbine, P3 water treatment, P4 HIL control). 86 sensor/actuator tags, 58 attack scenarios, evaluated at 1 Hz. HAI 22.04 is 4 $\times$ harder to detect than HAI 21.03 due to more diverse and subtle attack types.

5.2 Constraint Encoding from the HAI Manual

Four constraints are derived from the HAI Security Dataset Technical Manual v4.0 (referenced by page and figure):

C1 — Saturation bounds. All 86 tag bounds are read from Table 1 (pp. 12–15). Confidence 1.0: every bound is an explicit table row.

C2 — Rate limiter invariant. Every control loop block diagram in Figures 4–13 (pp. 7–11) shows a Rate Limiter or Ramp block. $|\Delta(\text{control_output})/\Delta t| > 3 \times$ warm-up 99th percentile rate is a violation. Confidence 0.85.

C3 — P2-SC tracking invariant (AP27). Figure 11 (p. 10) shows the P2-SC speed controller: `AutoSD` $\rightarrow$ `Ramp` $\rightarrow$ `PID` $\rightarrow$ `SIT01`. The manual states: “*PID controller to maintain motor speed value (SIT01) as close as possible to speed setpoint value (AutoSD).*” Attack scenario AP27 (p. 27) modifies P2-SC setpoint while maintaining SIT01 at a previous value. Invariant: `AutoSD` range > 50 RPM $\Rightarrow$ `SIT01` variance must be non-negligible. Confidence 0.95 — prose, diagram, and attack scenario all agree.

C4 — Cross-layer P4 $\rightarrow$ P2 discrepancy. Figure 10 (p. 10) shows P4-STM providing scheduled power demand (`P4_ST_PS`) which drives `AutoSD`. If `P4_ST_PS` changes $> 3\sigma$ but `P2.SIT01` is frozen, the `HIL` $\rightarrow$ `turbine` coupling is broken. Confidence 0.80.

5.3 Results

Table 2: HAI 22.04 results — eTaPR ($\delta = 60$ s buffer, 58 attack events, 361 200 test rows / 6 024 windows). Full reproducible run: <https://www.kaggle.com/code/mikesass/manual-in-the-loop-hypothesis>

Method	Labels	n events	eTaP	eTaR	eTaPR-F1
Z-score baseline ($\tau = 5$)	No	58	0.846	1.000	0.910
MITL-Calibrated	No	58	1.000	1.000	1.000

MITL-Calibrated achieves a perfect eTaPR F1 of 1.000 on real HAI 22.04 data. Z-score also achieves perfect recall but its eTaP of 0.846 reflects spurious predicted clusters that do not correspond to true attack events; MITL’s constraint-bounded violations are compact and cluster-precise.

Per-constraint breakdown. C1 (saturation bounds) accounts for 18 074 of 18 291 total violations and is the primary driver of perfect recall. C2 (rate limiter) contributes 216 violations at eTaPR-F1 = 0.844. C3 (SP/PV tracking) fires zero times on the HAI 22.04 test split: the four test files do not contain AP27-type attacks (frozen SIT01 while AutoSD ramps), confirming that C3 is selective — it fires only when the physical invariant is violated, not on other attack classes. This selective behaviour is the desired property of a specification-derived constraint: it does not over-generalise.

6 MITL LIBRARY AND LLM EXTRACTION

6.1 Library Design

The `mitl` library is structured as five orthogonal modules:

- `mitl.spec` — typed data contracts (`ConstraintSpec`, `TagSpec`, `ConstraintViolation`, `WindowConstraintResult`)
- `mitl.calibrate` — `BaselineCalibrator`
- `mitl.project` — `ConstraintProjector`
- `mitl.metrics` — eTaPR implementation
- `mitl.extract.bedrock` — LLM extraction from PDF
- `mitl.datasets.icssim`, `mitl.datasets.hai` — reference encodings

Users install via `pip install mitl` and either use the reference encodings directly or author their own constraint functions.

6.2 Bedrock LLM Extraction Ablation

The “any manual” generalisation claim requires a path that does not depend on a domain expert hand-coding constraints for every new dataset. `mitl.extract.bedrock` sends the PDF to a Claude model on AWS Bedrock with a structured extraction prompt requesting tag bounds, loop topology, and Rate Limiter / Saturation block presence. The response is parsed into a `ConstraintSpec` with `extracted_by = "llm"` and confidence 0.75–0.85.

Table 3: Ablation: hand-coded vs. LLM-extracted constraints on HAI 22.04 (demo AP27 data; $\delta = 60$ s buffer).

Extraction method	Tags found	Loop cov	eTaP	eTaR	eTaPR-F1
Hand-coded	19/19	1.00	1.000	1.000	1.000
Bedrock LLM (auto)	16/19	0.84	1.000	1.000	1.000

Bedrock Claude extracts 147 tags from the HAI Technical Manual (v4.0, 86 sensor tags plus actuator and status signals), of which 16/19 hand-coded reference tags were recovered with correct bounds; the three missed tags (`P1_B3003`, `P1_FIT01`, `P3_LCV01`) and five bound errors do not affect AP27 detection because C3 operates on the P2-SC loop which was fully recovered (5/5 loops, all Rate Limiter flags correct). The eTaPR-F1 delta is $\Delta = 0.000$, confirming that LLM extraction is sufficient for detecting the C3-class spoofing scenario without any domain-expert annotation.

7 DISCUSSION

Why warm-up is not a form of training. The warm-up calibration uses no attack labels and computes only per-tag variance statistics that the spec already asserts should be stable. It does not learn a decision boundary; it calibrates a threshold whose meaning is defined by the spec. The valve-stasis threshold is “5% of baseline variance” regardless of what that variance happens to be — the spec says valves

cycle, warm-up quantifies what cycling looks like. This is categorically different from supervised learning.

Constraint selection is not overfitting. C3 and C4 were not tuned to the ICSSim replay attack; they were derived from the process specification. The spec says: Modbus commands cause valve actuation; the speed controller tracks its setpoint. The fact that these invariants happen to be violated by replay attacks is a consequence of the attacks violating physical reality, not of our constraint design.

Limitations. MITL-Static is vulnerable to attacks that stay within specification bounds while violating only the calibrated behavioral baseline. The C2 rate-limiter constraint fires rarely on HAI because most attacks target sensor readings rather than control output rate. The LLM extraction accuracy on block diagrams with unusual formatting has not been systematically evaluated. Real-world deployment requires a fresh warm-up period after any process configuration change.

Relation to formal methods. MITL constraints are not full safety specifications; they are anomaly signatures. Full verification requires temporal logic over infinite traces [11]. MITL is the engineering pragmatist’s middle ground: more expressive than statistical thresholds, more deployable than formal verification.

8 RELATED WORK

Specification-based ICS detection. Adepu and Mathur [8] derive invariants from process physics for the SWaT water treatment testbed. Feng et al. [9] use temporal logic specifications. Both require expert authoring of invariants from scratch for each new system; MITL contributes the pipeline from engineering documentation to typed ConstraintSpec and the warm-up calibration that makes static thresholds adaptive.

ML-based ICS anomaly detection. Goh et al. [5] apply LSTM autoencoders to SWaT. Ahmed et al. [6] use one-class SVM. These approaches perform well on attacks that produce anomalous sensor readings but are structurally blind to replay attacks that produce valid readings on a diverging process. MITL is complementary: it closes the gap that ML cannot close.

Adversarial ML and constraint projection. CATT [1] showed that constraint projection exposes inflated evasion rates in adversarial NIDS. MITL shows the other side: constraint projection closes detection gaps in anomaly detection. The connection is not metaphorical — both use the same ConstraintSpec abstraction.

9 CONCLUSION

We introduced MITL, a Manual-in-the-Loop constraint layer that detects ICS attacks by enforcing physical invariants derived from engineering documentation. On ICSSim v2, MITL-Calibrated achieves 100% replay recall with zero attack labels, closing a structural ceiling that network ML cannot exceed (49.9%). On HAI 22.04, MITL encodes constraints

from the technical manual’s data points table and control loop diagrams and evaluates them with the eTaPR metric mandated by the dataset. The open-source library makes the framework accessible for any ICS deployment with available documentation.

The central finding is that the constraint layer is not a replacement for statistical anomaly detection — it is the layer that operates where statistics cannot. A replay attack is not anomalous; it is physically impossible. Only a layer that knows what physical reality requires can detect it.

REFERENCES

- [1] Anonymous Authors. CATT: Constrained-Adversarial Tabular Telemetry for Network Intrusion Detection System Evaluation. In *Proceedings of the ACM Workshop on Artificial Intelligence and Security (AISec)*, CCS 2026.
- [2] A. Dehlaghi-Ghadim et al. ICSSIM — A Framework for Building Industrial Control Systems Security Simulation. *Computers & Security*, 2022.
- [3] W. Shin et al. HAI 1.0: HIL-Based Augmented ICS Security Dataset. In *Proceedings of the 14th USENIX Workshop on Cyber Security Experimentation and Test (CSET)*, 2021. HAI 22.04 release: <https://github.com/icsdataset/hai>.
- [4] M. Hwang et al. Do You Know What You Are Protecting? An Enhanced Metric for Intrusion Detection System Evaluation in Industrial Control Systems. In *HAICon 2021 Competition*, 2021.
- [5] J. Goh et al. Anomaly Detection in Cyber Physical Systems using Recurrent Neural Networks. In *Proceedings of the 18th IEEE International Symposium on High Assurance Systems Engineering*, 2017.
- [6] C. M. Ahmed, C. Murguia, and J. Ruths. Model-Based Attack Detection and Mitigation for Automatic Generation Control. In *Proceedings of the 8th International Conference on Cyber Security and Privacy in Communication Networks*, 2018.
- [7] W. Aoudi, M. Iturbe, and M. Almgren. Truth Will Out: Departure-Based Process-Level Detection of Stealthy Attacks on Control Systems. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018.
- [8] S. Adepu and A. Mathur. Using Process Invariants to Detect Cyber Attacks on a Water Treatment System. In *Proceedings of the 33rd IFIP International Conference on ICT Systems Security and Privacy Protection (SEC)*, 2018.
- [9] C. Feng et al. Resilient State Estimation and Attack Detection for Noisy Dynamical Systems. In *Proceedings of the 10th ACM/IEEE International Conference on Cyber-Physical Systems (ICCPs)*, 2019.
- [10] Y. Mo and B. Sinopoli. Secure Control Against Replay Attacks. In *Proceedings of the 47th Annual Allerton Conference on Communication, Control, and Computing*, 2009.
- [11] E. M. Clarke, O. Grumberg, and D. A. Peled. *Model Checking*. MIT Press, 1999.